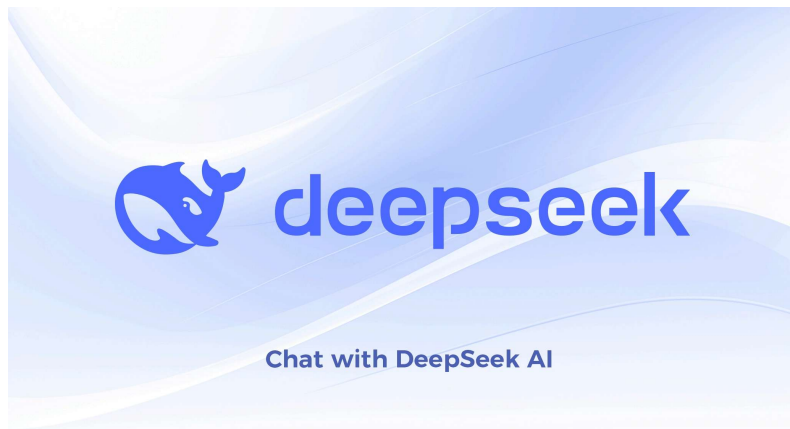


# Mongoose ODM Library Explanation - DeepSeek

---



Here's a comprehensive guide to **Mongoose Schema Type Options** - configuring how fields behave, are validated, and transformed.

## Quick Answer: Basic Type Options

---

javascript

```
const userSchema = new mongoose.Schema({
  name: {
    type: String,
    required: true,
    trim: true,
    lowercase: true
  },
  age: {
    type: Number,
    min: 0,
    max: 120,
    default: 18
  }
});
```

## Complete Schema Type Options

---

### 1. All Schema Types Common Options

---

javascript

```
const schema = new mongoose.Schema({
  field: {
    type: String,

    required: [true, 'Field is required'],
    validate: function(value) { return true; },

    default: 'default value',
    default: function() { return Date.now(); },

    index: true,
    unique: true,
    sparse: true,

    get: function(value) { return value; },
    set: function(value) { return value; },

    alias: 'alternateName',
    immutable: true,
    select: false,
    hide: true,
    _id: false
  }
});
```

## 2. String Type Options

---

javascript

```
const stringSchema = new mongoose.Schema({
  username: {
    type: String,

    required: [true, 'Username is required'],
    default: 'guest',

    lowercase: true,
    uppercase: true,
    trim: true,

    minlength: [3, 'Must be at least 3 characters'],
    maxlength: [50, 'Cannot exceed 50 characters'],

    match: [/^[a-zA-Z0-9]+$/, 'Only letters and numbers
allowed'],

    enum: {
      values: ['user', 'admin', 'moderator'],
      message: '{VALUE} is not a valid role'
    },

    validate: {
      validator: function(v) {
        return v.length > 0;
      },
      message: 'Cannot be empty'
    },

    index: true,
    unique: true,

    get: v => v?.toUpperCase(),
    set: v => v?.toLowerCase(),

    immutable: true,
    select: false
  },

  firstName: String,
```

```
lastName: { type: String, required: true }  
});
```

## String Practical Examples

---

javascript

```
email: {
  type: String,
  required: [true, 'Email is required'],
  unique: true,
  lowercase: true,
  trim: true,
  match: [
    /^^\w+([\.-]?\w+)*@\w+([\.-]?\w+)*(\.\w{2,3})+$/ ,
    'Please enter a valid email'
  ],
  index: true,
  get: email => email?.toLowerCase(),
  set: email => email?.toLowerCase().trim()
},

website: {
  type: String,
  trim: true,
  lowercase: true,
  validate: {
    validator: function(v) {
      try {
        new URL(v);
        return true;
      } catch(e) {
        return false;
      }
    },
    message: 'Invalid URL'
  },
  set: function(url) {
    if (!url.startsWith('http')) {
      url = 'https://' + url;
    }
    return url;
  }
},

phone: {
  type: String,
  trim: true,
  validate: {
    validator: function(v) {
      return /^^\+?[\d\s-]{10,}$/.test(v);
    },
    message: 'Invalid phone number'
  },
}
```

```
set: function(v) {  
  
    return v.replace(/^[^d+]/g, '');  
}  
}
```

### 3. Number Type Options

---

javascript

```
const numberSchema = new mongoose.Schema({
  age: {
    type: Number,

    required: [true, 'Age is required'],
    default: 18,

    min: [0, 'Age cannot be negative'],
    max: [150, 'Age cannot exceed 150'],

    validate: {
      validator: Number.isInteger,
      message: '{VALUE} must be an integer'
    },

    index: true,
    unique: false,

    get: v => Math.round(v),
    set: v => Math.round(v),

    immutable: true,
    select: false
  },

  price: {
    type: Number,
    required: true,
    min: 0.01,
    max: 999999.99,
    get: v => (v / 100).toFixed(2),
    set: v => Math.round(v * 100)
  },

  score: {
    type: Number,
    default: 0,
    min: 0,
    max: 100
  }
});
```

## Number Practical Examples

---

javascript



```
percentage: {
  type: Number,
  min: 0,
  max: 100,
  default: 0,
  validate: {
    validator: function(v) {
      return v >= 0 && v <= 100;
    },
    message: 'Percentage must be between 0 and 100'
  },
  get: v => `${v}%`,
  set: v => parseFloat(v)
},

rating: {
  type: Number,
  min: 1,
  max: 5,
  default: 5,
  validate: {
    validator: Number.isInteger,
    message: 'Rating must be a whole number'
  },
  get: v => v.toFixed(1),
  set: v => Math.round(v)
},

latitude: {
  type: Number,
  required: true,
  min: -90,
  max: 90,
  validate: {
    validator: function(v) {
      return Math.abs(v) <= 90;
    },
    message: 'Invalid latitude'
  }
},

longitude: {
  type: Number,
  required: true,
  min: -180,
  max: 180,
  validate: {
    validator: function(v) {
```

```
    return Math.abs(v) <= 180;
  },
  message: 'Invalid longitude'
}
```

## 4. Date Type Options

---

javascript

```
const dateSchema = new mongoose.Schema({
  birthDate: {
    type: Date,

    required: [true, 'Birth date is required'],
    default: Date.now,
    default: new Date(),

    min: [new Date('1900-01-01'), 'Date too old'],
    max: [new Date(), 'Date cannot be in the future'],

    validate: {
      validator: function(v) {
        return v <= new Date();
      },
      message: 'Date cannot be in the future'
    },

    index: true,

    get: v => v?.toISOString().split('T')[0],
    set: v => new Date(v),

    immutable: true,
    select: false
  },

  lastLogin: {
    type: Date,
    default: Date.now
  },

  expiresAt: {
    type: Date,
    index: { expires: '1h' }
  }
});

const userSchema = new mongoose.Schema({
  name: String
}, {
```

```
timestamps: {
  createdAt: 'created_at',
  updatedAt: 'updated_at'
}
});
```

## Date Practical Examples

---

javascript

```
eventDate: {
  type: Date,
  required: true,
  validate: {
    validator: function(v) {
      return v > new Date();
    },
    message: 'Event date must be in the future'
  },
  get: v => v.toLocaleDateString('en-US'),
  set: v => new Date(v)
},

age: {
  type: Number,
  get: function() {
    if (!this.birthDate) return null;
    const today = new Date();
    const age = today.getFullYear() -
this.birthDate.getFullYear();
    const monthDiff = today.getMonth() -
this.birthDate.getMonth();
    if (monthDiff < 0 || (monthDiff === 0 && today.getDate() <
this.birthDate.getDate())) {
      return age - 1;
    }
    return age;
  }
}
```

## 5. Array Type Options

---

javascript

```
const arraySchema = new mongoose.Schema({

  tags: {
    type: [String],

    default: [],

    validate: {
      validator: function(v) {
        return v.length <= 10;
      },
      message: 'Maximum 10 tags allowed'
    },

    index: true,

    get: v => v.sort(),
    set: v => [...new Set(v)]
  },

  scores: {
    type: [Number],
    validate: {
      validator: function(v) {
        return v.every(score => score >= 0 && score <= 100);
      },
      message: 'All scores must be between 0 and 100'
    },
    default: []
  },

  addresses: [{
    street: String,
    city: String,
    isPrimary: { type: Boolean, default: false }
  }],

  metadata: {
    type: [],
    default: []
  }
});
```

## Array Practical Examples

---

javascript

```
uniqueTags: {
  type: [String],
  set: function(v) {
    return [...new Set(v)];
  },
  validate: {
    validator: function(v) {
      return v.length === new Set(v).size;
    },
    message: 'Duplicate values not allowed'
  }
},

requirements: {
  type: [String],
  validate: {
    validator: function(v) {
      return v.length >= 3;
    },
    message: 'At least 3 items required'
  }
},

matrix: {
  type: [[Number]],
  validate: {
    validator: function(v) {
      return v.every(row => row.length === v[0]?.length);
    },
    message: 'Matrix must be rectangular'
  }
}
```

## 6. ObjectId Type Options

---

javascript

```
const objectIdSchema = new mongoose.Schema({
  userId: {
    type: mongoose.Schema.Types.ObjectId,

    required: true,

    ref: 'User',

    index: true,
    unique: true,

    default: () => new mongoose.Types.ObjectId(),

    validate: {
      validator: mongoose.Types.ObjectId.isValid,
      message: 'Invalid ObjectId'
    },

    get: v => v.toString(),
    set: v => v
  },

  author: {
    type: mongoose.Schema.Types.ObjectId,
    ref: 'User',
    required: true,
    autopopulate: true
  },

  followers: [{
    type: mongoose.Schema.Types.ObjectId,
    ref: 'User',
    index: true
  }]
});
```

## 7. Mixed Type Options (Any Type)

---

javascript

```
const mixedSchema = new mongoose.Schema({

  metadata: {
    type: mongoose.Schema.Types.Mixed,

    required: false,
    default: {},

    get: v => v || {},
    set: v => ({ ...v })
  },

  settings: {
    type: mongoose.Schema.Types.Mixed,
    validate: {
      validator: function(v) {
        return typeof v === 'object';
      },
      message: 'Settings must be an object'
    }
  }
});

const doc = new Model({
  metadata: {
    theme: 'dark',
    preferences: { notifications: true },
    anyKey: 'any value',
    nested: { arbitrary: 'structure' }
  }
});

doc.metadata.newField = 'value';
doc.markModified('metadata');
await doc.save();
```

## 8. Map Type Options

---

javascript



```
const mapSchema = new mongoose.Schema({

  translations: {
    type: Map,
    of: String,
    default: () => new Map(),

    required: false,

    get: v => Object.fromEntries(v),
    set: v => new Map(Object.entries(v))
  },

  scores: {
    type: Map,
    of: Number,
    default: {},
    validate: {
      validator: function(v) {
        const scores = Array.from(v.values());
        return scores.every(score => score >= 0);
      },
      message: 'Scores must be non-negative'
    }
  },

  config: {
    type: Map,
    of: mongoose.Schema.Types.Mixed,
    default: {}
  }
});

const doc = new Model();
doc.translations.set('en', 'Hello');
doc.translations.set('es', 'Hola');
console.log(doc.translations.get('en'));
```

## 9. Buffer Type Options (Binary Data)

---

javascript

```
const bufferSchema = new mongoose.Schema({

  avatar: {
    type: Buffer,

    required: false,

    validate: {
      validator: function(v) {
        return v.length <= 5 * 1024 * 1024;
      },
      message: 'File too large (max 5MB)'
    },

    get: v => v ? v.toString('base64') : null,

    set: v => {
      if (typeof v === 'string') {
        return Buffer.from(v, 'base64');
      }
      return v;
    }
  },

  fileData: {
    type: Buffer,
    validate: {
      validator: function(v) {
        return v.length <= 16 * 1024 * 1024;
      },
      message: 'File exceeds MongoDB document limit'
    }
  }
});
```

## 10. Decimal128 Type Options

---

javascript

```
const decimalSchema = new mongoose.Schema({

  price: {
    type: mongoose.Schema.Types.Decimal128,

    required: [true, 'Price is required'],
    default: 0.00,

    min: 0,
    validate: {
      validator: function(v) {
        return v.toString().split('.')[1]?.length <= 2;
      },
      message: 'Maximum 2 decimal places'
    },

    get: v => parseFloat(v.toString()),

    set: v =>
mongoose.Types.Decimal128.fromString(v.toString())
  },

  pi: {
    type: mongoose.Schema.Types.Decimal128,
    default: () =>
mongoose.Types.Decimal128.fromString('3.141592653589793')
  }
});
```

## 11. UUID Type Options

---

javascript

```
const uuidSchema = new mongoose.Schema({

  sessionId: {
    type: mongoose.Schema.Types.UUID,

    default: () => crypto.randomUUID(),

    required: true,
    unique: true,
    index: true,

    get: v => v.toString(),
    set: v => v
  },

  apiKey: {
    type: mongoose.Schema.Types.UUID,
    default: () => crypto.randomUUID(),
    unique: true,
    index: true,
    immutable: true,
    select: false
  }
});
```

## 12. Combined Practical Examples

---

### Example 1: Complete User Schema

---

javascript

```
const userSchema = new mongoose.Schema({

  username: {
    type: String,
    required: [true, 'Username required'],
    unique: true,
    trim: true,
    lowercase: true,
    minlength: [3, 'Minimum 3 characters'],
    maxlength: [20, 'Maximum 20 characters'],
    match: [/^[a-z0-9_]+$/, 'Only letters, numbers,
underscore'],
    index: true,
    get: v => v,
    set: v => v.toLowerCase().trim()
  },

  email: {
    type: String,
    required: true,
    unique: true,
    lowercase: true,
    trim: true,
    match: [/^\S+@\S+\.\S+$/, 'Invalid email'],
    index: true,
    select: true
  },

  password: {
    type: String,
    required: true,
    minlength: 8,
    select: false,
    validate: {
      validator: function(v) {
        return /^(?=.*[a-z])(?=.*[A-Z])(?=.*\d)/.test(v);
      },
      message: 'Password must contain uppercase, lowercase,
and number'
    }
  },

  age: {
    type: Number,
    min: 13,
    max: 120,
    validate: {
```

```
      validator: Number.isInteger,
      message: 'Age must be integer'
    }
  },

  lastLogin: {
    type: Date,
    default: Date.now,
    get: v => v?.toLocaleString(),
    set: v => v || new Date()
  },

  role: {
    type: mongoose.Schema.Types.ObjectId,
    ref: 'Role',
    required: true
  },

  interests: {
    type: [String],
    default: [],
    validate: {
      validator: function(v) {
        return v.length <= 10;
      },
      message: 'Maximum 10 interests'
    },
    set: v => [...new Set(v)]
  },

  preferences: {
    type: mongoose.Schema.Types.Mixed,
    default: {
      theme: 'light',
      notifications: true
    }
  },

  nameTranslations: {
    type: Map,
    of: String,
    default: new Map()
  },

  createdAt: {
```

```

    type: Date,
    default: Date.now,
    immutable: true,
    get: v => v?.toISOString()
  },

  updatedAt: {
    type: Date,
    default: Date.now,
    get: v => v?.toISOString()
  }
}, {

  timestamps: {
    createdAt: 'created_at',
    updatedAt: 'updated_at'
  },
  toJSON: {
    virtuals: true,
    getters: true,
    transform: function(doc, ret) {
      delete ret.__v;
      return ret;
    }
  },
  toObject: { virtuals: true, getters: true },
  id: true,
  strict: true,
  bufferCommands: true,
  autoIndex: process.env.NODE_ENV !== 'production'
});

userSchema.virtual('profileUrl').get(function() {
  return `/users/${this._id}`;
});

userSchema.pre('save', function(next) {
  this.updatedAt = new Date();
  next();
});

```

## Example 2: Product Schema with Advanced Options

javascript

```
const productSchema = new mongoose.Schema({
  sku: {
    type: String,
    required: true,
    unique: true,
    uppercase: true,
    trim: true,
    match: [/^[A-Z0-9]{8}$/, 'SKU must be 8 alphanumeric
characters'],
    index: true,
    immutable: true
  },

  name: {
    type: String,
    required: [true, 'Product name required'],
    trim: true,
    maxlength: [200, 'Name too long'],
    text: true,
    index: true
  },

  description: {
    type: String,
    trim: true,
    maxlength: 5000,
    select: false
  },

  price: {
    type: mongoose.Schema.Types.Decimal128,
    required: true,
    min: 0,
    get: v => parseFloat(v.toString()),
    set: v =>
mongoose.Types.Decimal128.fromString(v.toString())
  },

  quantity: {
    type: Number,
    required: true,
    min: 0,
    max: 999999,
    default: 0
  },

  categories: [{
    type: mongoose.Schema.Types.ObjectId,
    ref: 'Category',
    index: true
  ]
})
```



```
  ]],

  tags: {
    type: [String],
    index: true,
    set: v => v.map(tag => tag.toLowerCase().trim()),
    validate: {
      validator: function(v) {
        return v.length <= 20;
      },
      message: 'Maximum 20 tags'
    }
  },

  specifications: {
    type: Map,
    of: mongoose.Schema.Types.Mixed,
    default: new Map()
  },

  images: [{
    url: {
      type: String,
      validate: {
        validator: v => /^https?:\\/\\/\\.test(v),
        message: 'Invalid image URL'
      }
    },
    alt: String,
    isPrimary: { type: Boolean, default: false }
  }],

  metadata: {
    createdBy: {
      type: mongoose.Schema.Types.ObjectId,
      ref: 'User',
      immutable: true
    },
    views: {
      type: Number,
      default: 0,
      min: 0
    },
    rating: {
      type: Number,
      min: 0,
      max: 5,
      default: 0,
      get: v => v.toFixed(1),
      set: v => parseFloat(v)
    }
  }
}
```

```

    },

    saleEndDate: {
      type: Date,
      index: { expires: 0 }
    }
  }, {
    timestamps: true,
    toJSON: {
      getters: true,
      virtuals: true,
      transform: (doc, ret) => {
        ret.price = ret.price?.toString();
        return ret;
      }
    }
  }
});

productSchema.index({ name: 'text', description: 'text' });
productSchema.index({ price: 1, quantity: -1 });

```

## Quick Reference Table

---

### Best Practices

---

1. **Use appropriate types** - Don't use Mixed when a specific type works
2. **Set `select: false`** for sensitive data (passwords, tokens)
3. **Use `immutable: true`** for fields that shouldn't change (createdAt, ID fields)
4. **Add indexes** with `index: true` for frequently queried fields
5. **Use `trim: true`** for all string fields
6. **Set `required` with custom messages** for better error handling
7. **Use getters/setters** for data transformation (currency, dates)
8. **Avoid `unique: true` on large text fields** - use hashed indexes
9. **Use Decimal128 for currency** instead of Number
10. **Add `validate` for complex business rules**

Need help with specific type options or advanced configurations?

validation in updation and error

Here's a comprehensive guide to **validation during update operations** and **error handling** in Mongoose.

## Quick Answer: Update Validation

---

javascript

```
const result = await User.updateOne(
  { _id: userId },
  { $set: { age: 200 } },
  { runValidators: true }
);

const user = await User.findOneAndUpdate(
  { email: 'john@example.com' },
  { age: 200 },
  {
    new: true,
    runValidators: true,
    context: 'query'
  }
);
```

## 1. Why Validation Doesn't Run by Default on Updates

---

javascript

```
const userSchema = new mongoose.Schema({
  name: { type: String, required: true, minlength: 2 },
  age: { type: Number, min: 0, max: 120 }
});

await User.updateOne(
  { _id: userId },
  { $set: { name: 'J', age: 200 } }
);

await User.updateOne(
  { _id: userId },
  { $set: { name: 'J', age: 200 } },
  { runValidators: true }
);
```

## 2. Update Methods with Validation

---

### updateOne() with Validation

---

javascript

```
try {
  const result = await User.updateOne(
    { email: 'john@example.com' },
    {
      $set: {
        name: 'Jo',
        age: 25
      }
    },
    {
      runValidators: true,
      context: 'query'
    }
  );

  console.log(result);

} catch (error) {
  console.error(error);
}
```

## updateMany() with Validation

---

javascript

```
try {
  const result = await User.updateMany(
    { role: 'guest' },
    { $set: { age: -5 } },
    { runValidators: true }
  );
  console.log(`Updated ${result.modifiedCount} users`);
} catch (error) {
  console.error('Validation failed for some users:',
    error.message);
}
```

## findOneAndUpdate() with Validation

---

javascript

```
const user = await User.findOneAndUpdate(
  { email: 'john@example.com' },
  { $set: { age: 200 } },
  {
    new: true,
    runValidators: true,
    context: 'query'
  }
);
```

## findByIdAndUpdate() with Validation

---

javascript

```
try {
  const user = await User.findByIdAndUpdate(
    userId,
    { $set: { name: 'J', age: 25 } },
    {
      new: true,
      runValidators: true,
      context: 'query'
    }
  );
} catch (error) {

  console.error(error.errors.name.message);
}
```

### 3. Complete Update Validation Example

---

javascript

```
const mongoose = require('mongoose');

const userSchema = new mongoose.Schema({
  name: {
    type: String,
    required: [true, 'Name is required'],
    minlength: [3, 'Name must be at least 3 characters'],
    maxlength: [50, 'Name cannot exceed 50 characters']
  },
  email: {
    type: String,
    required: [true, 'Email is required'],
    unique: true,
    lowercase: true,
    match: [/^\S+@\S+\.\S+$/, 'Invalid email format']
  },
  age: {
    type: Number,
    min: [0, 'Age cannot be negative'],
    max: [150, 'Age cannot exceed 150']
  },
  role: {
    type: String,
    enum: {
      values: ['user', 'admin', 'moderator'],
      message: '{VALUE} is not a valid role'
    }
  },
  password: {
    type: String,
    required: true,
    minlength: [6, 'Password too short'],
    select: false
  }
});

const User = mongoose.model('User', userSchema);

async function safeUpdateUser(userId, updateData) {
  try {
    const user = await User.findByIdAndUpdate(
      userId,
      { $set: updateData },
      {
        new: true,
        runValidators: true,
        context: 'query'
      }
    );
  }
}
```

```
);

if (!user) {
  return { success: false, error: 'User not found' };
}

return { success: true, data: user };

} catch (error) {

  if (error.name === 'ValidationError') {
    const errors = {};
    for (let field in error.errors) {
      errors[field] = error.errors[field].message;
    }
    return { success: false, validationErrors: errors };
  }

  if (error.code === 11000) {
    return { success: false, error: 'Duplicate key error' };
  }

  return { success: false, error: error.message };
}

}

const result = await safeUpdateUser(userId, {
  name: 'Jo',
  age: 200,
  role: 'superuser'
});

if (!result.success) {
  console.log('Validation errors:', result.validationErrors);
}

}
```

## 4. Handling Different Update Scenarios

---

### Scenario 1: Partial Updates

---

javascript



```
async function updateUserPartial(userId, updates) {

  const filteredUpdates = {};
  const allowedFields = ['name', 'email', 'age', 'role'];

  for (const field of allowedFields) {
    if (updates[field] !== undefined) {
      filteredUpdates[field] = updates[field];
    }
  }

  try {
    const user = await User.findByIdAndUpdate(
      userId,
      { $set: filteredUpdates },
      {
        new: true,
        runValidators: true,
        context: 'query'
      }
    );
    return user;
  } catch (error) {
    if (error.name === 'ValidationError') {

      const errors = {};
      for (let field in error.errors) {
        if (filteredUpdates[field]) {
          errors[field] = error.errors[field].message;
        }
      }
      throw new Error(JSON.stringify(errors));
    }
    throw error;
  }
}
```

## Scenario 2: Conditional Updates with Validation

---

javascript

```
async function conditionalUpdate(userId, updateData,
condition) {
  const query = { _id: userId, ...condition };

  const user = await User.findOneAndUpdate(
    query,
    { $set: updateData },
    {
      new: true,
      runValidators: true,
      context: 'query'
    }
  );

  if (!user) {
    throw new Error('User not found or conditions not met');
  }

  return user;
}

await conditionalUpdate(
  userId,
  { age: 30 },
  { isActive: true }
);
```

### Scenario 3: Upsert with Validation

---

typescript

```
async function upsertWithValidation(email, userData) {
  const result = await User.findOneAndUpdate(
    { email: email },
    { $set: userData },
    {
      upsert: true,
      new: true,
      runValidators: true,
      setDefaultsOnInsert: true,
      context: 'query'
    }
  );

  return result;
}

const user = await upsertWithValidation('new@example.com', {
  name: 'Ne',
  age: 25
});
```

## 5. Common Validation Errors on Update

---

### ValidationError Structure

---

javascript

```
try {
  await User.findByIdAndUpdate(
    userId,
    { $set: { name: 'J', age: -5, email: 'invalid' } },
    { runValidators: true, new: true }
  );
} catch (error) {
  if (error.name === 'ValidationError') {
    console.log('Error name:', error.name);
    console.log('Error message:', error.message);

    for (let field in error.errors) {
      console.log(`Field: ${field}`);
      console.log(`  Message:
${error.errors[field].message}`);
      console.log(`  Value: ${error.errors[field].value}`);
      console.log(`  Kind: ${error.errors[field].kind}`);
      console.log(`  Path: ${error.errors[field].path}`);
    }

    const messages = Object.values(error.errors).map(e =>
e.message);
    console.log('All errors:', messages);
  }
}
```

## Handling Specific Validation Types

---

javascript

```
try {
  await User.updateOne(
    { _id: userId },
    { $set: { email: 'existing@example.com' } },
    { runValidators: true }
  );
} catch (error) {
  switch (error.name) {
    case 'ValidationError':

      console.error('Validation error:', error.message);
      break;

    case 'MongoServerError':
      if (error.code === 11000) {

        console.error('Duplicate field value:',
error.keyValue);
      }
      break;

    case 'CastError':

      console.error('Cast error:', error.message);
      break;

    default:
      console.error('Unknown error:', error);
  }
}
```

## 6. Custom Validators in Updates

---

### Async Validator in Update

---

javascript

```
const productSchema = new mongoose.Schema({
  name: String,
  sku: {
    type: String,
    unique: true,
    validate: {
      validator: async function(sku) {

        const existing = await this.constructor.findOne({ sku
      });

      if (existing && !existing._id.equals(this._id)) {
        return false;
      }
      return true;
    },
    message: 'SKU already exists'
  }
});

const product = await Product.findOneAndUpdate(
  { _id: productId },
  { $set: { sku: 'NEW-SKU' } },
  {
    runValidators: true,
    context: 'query'
  }
);
```

## Cross-Field Validation in Updates

---

javascript

```
const eventSchema = new mongoose.Schema({
  startDate: Date,
  endDate: Date,
  validate: {
    validator: function() {
      if (this.startDate && this.endDate) {
        return this.endDate > this.startDate;
      }
      return true;
    },
    message: 'End date must be after start date'
  }
});

const event = await Event.findOneAndUpdate(
  { _id: eventId },
  {
    $set: {
      startDate: new Date('2024-01-01'),
      endDate: new Date('2023-12-31')
    }
  },
  {
    runValidators: true,
    context: 'query'
  }
);
```

## 7. Error Handling Patterns

---

### Pattern 1: Centralized Error Handler

---

javascript

```
class UpdateErrorHandler {
  static handle(error, context = {}) {

    console.error('Update error:', {
      error: error.message,
      stack: error.stack,
      context
    });

    if (error.name === 'ValidationError') {
      const fields = {};
      for (let field in error.errors) {
        fields[field] = error.errors[field].message;
      }
      return {
        status: 400,
        type: 'VALIDATION_ERROR',
        message: 'Validation failed',
        fields
      };
    }

    if (error.code === 11000) {
      const field = Object.keys(error.keyPattern)[0];
      return {
        status: 409,
        type: 'DUPLICATE_ERROR',
        message: `${field} already exists`,
        field
      };
    }

    if (error.name === 'CastError') {
      return {
        status: 400,
        type: 'INVALID_ID',
        message: `Invalid ${error.path}: ${error.value}`
      };
    }

    if (error.name === 'DocumentNotFoundError') {
      return {
        status: 404,
        type: 'NOT_FOUND',
        message: 'Document not found'
      };
    }
  }
}
```



```
    }

    return {
      status: 500,
      type: 'INTERNAL_ERROR',
      message: 'An unexpected error occurred'
    };
  }
}

app.put('/api/users/:id', async (req, res) => {
  try {
    const user = await User.findByIdAndUpdate(
      req.params.id,
      { $set: req.body },
      {
        new: true,
        runValidators: true,
        context: 'query'
      }
    );

    res.json({ success: true, data: user });

  } catch (error) {
    const handled = UpdateErrorHandler.handle(error, { userId:
req.params.id });
    res.status(handled.status).json(handled);
  }
});
```

## Pattern 2: Retry on Validation Error

---

javascript

```
async function updateWithRetry(userId, updateData, maxRetries
= 3) {
  let lastError;

  for (let attempt = 1; attempt <= maxRetries; attempt++) {
    try {
      const user = await User.findByIdAndUpdate(
        userId,
        { $set: updateData },
        {
          new: true,
          runValidators: true,
          context: 'query'
        }
      );

      return { success: true, data: user };

    } catch (error) {
      lastError = error;

      if (error.name === 'ValidationError') {
        throw error;
      }

      if (error.code === 11000) {
        throw error;
      }

      console.log(`Attempt ${attempt} failed. Retrying...`);
      await new Promise(resolve => setTimeout(resolve, 1000 *
attempt));
    }

    throw lastError;
  }
}
```

### Pattern 3: Bulk Update with Partial Success

---

javascript

```
async function bulkUpdateWithValidation(updates) {
  const results = {
    successful: [],
    failed: [],
    summary: {
      total: updates.length,
      succeeded: 0,
      failed: 0
    }
  };

  for (const update of updates) {
    try {
      const user = await User.findOneAndUpdate(
        { email: update.email },
        { $set: update.data },
        {
          new: true,
          runValidators: true,
          context: 'query'
        }
      );

      results.successful.push({
        email: update.email,
        data: user
      });
      results.summary.succeeded++;

    } catch (error) {
      results.failed.push({
        email: update.email,
        error: error.name === 'ValidationError'
          ? Object.values(error.errors).map(e => e.message)
          : error.message
      });
      results.summary.failed++;
    }
  }

  return results;
}

const updates = [
  { email: 'user1@example.com', data: { name: 'User 1', age: 25 } },
  { email: 'user2@example.com', data: { name: 'U', age: 30 } },
  { email: 'user3@example.com', data: { name: 'User 3', age: 30 } }
```

```
-5 } }
```

```
];
```

```
const result = await bulkUpdateWithValidation(updates);  
console.log(`Succeeded: ${result.summary.succeeded}, Failed:  
${result.summary.failed}`);
```

## 8. Transaction with Validation

---

javascript

```
async function updateWithTransaction(userId, updates) {
  const session = await mongoose.startSession();
  session.startTransaction();

  try {

    const user = await User.findByIdAndUpdate(
      userId,
      { $set: updates },
      {
        session,
        new: true,
        runValidators: true,
        context: 'query'
      }
    );

    if (!user) {
      throw new Error('User not found');
    }

    await Profile.findOneAndUpdate(
      { userId: userId },
      { $set: { lastUpdated: new Date() } },
      { session, runValidators: true }
    );

    await session.commitTransaction();
    return { success: true, data: user };

  } catch (error) {
    await session.abortTransaction();

    if (error.name === 'ValidationError') {
      return {
        success: false,
        error: 'Validation failed',
        details: error.errors
      };
    }

    return { success: false, error: error.message };

  } finally {
    session.endSession();
  }
}
```

## 9. Common Pitfalls and Solutions

---

### Pitfall 1: Forgetting runValidators

---

javascript

```
await User.findByIdAndUpdate(userId, { age: 200 });

await User.findByIdAndUpdate(
  userId,
  { age: 200 },
  { runValidators: true }
);
```

### Pitfall 2: Missing context for Async Validators

---

javascript

```
await User.findOneAndUpdate(
  { email: 'john@example.com' },
  { email: 'new@example.com' },
  { runValidators: true }
);

await User.findOneAndUpdate(
  { email: 'john@example.com' },
  { email: 'new@example.com' },
  {
    runValidators: true,
    context: 'query'
  }
);
```

### Pitfall 3: Partial Updates and Required Fields

---

javascript

```
const schema = new mongoose.Schema({
  name: { type: String, required: true },
  email: { type: String, required: true }
});

await User.findByIdAndUpdate(
  userId,
  { $set: { email: 'new@example.com' } },
  { runValidators: true }
);

await User.findByIdAndUpdate(
  userId,
  { $set: { email: 'new@example.com' } },
  {
    runValidators: true,
    context: 'query'
  }
);
```

## Pitfall 4: Validators on nested paths

---

javascript

```
const schema = new mongoose.Schema({
  address: {
    street: { type: String, required: true },
    city: { type: String, required: true }
  }
});

await User.updateOne(
  { _id: userId },
  { $set: { 'address.city': 'Boston' } },
  { runValidators: true }
);
```

## 10. Debugging Update Validation

---

javascript

```
mongoose.set('debug', true);

userSchema.pre('findOneAndUpdate', function() {
  console.log('Update filter:', this.getFilter());
  console.log('Update operation:', this.getUpdate());
  console.log('Options:', this.options);
});

try {
  await User.findByIdAndUpdate(userId, update, {
    runValidators: true });
} catch (error) {
  if (error.name === 'ValidationError') {
    console.error('Validation error details:');
    Object.keys(error.errors).forEach(key => {
      console.error(`- ${key}: ${error.errors[key].message}`);
      console.error(`  Value: ${error.errors[key].value}`);
      console.error(`  Type: ${error.errors[key].kind}`);
    });
  }
}
```

## Quick Reference

---

### Best Practices

---

1. **Always use `runValidators: true`** on update operations
2. **Use `context: 'query'`** for async validators
3. **Handle `ValidationError` specifically** - it's different from MongoDB errors
4. **Use `transactions`** when updating multiple related documents
5. **Log validation errors** with field details for debugging
6. **Return meaningful error messages** to API clients
7. **Validate input at application level** before passing to Mongoose
8. **Use schema validation as last line of defense**
9. **Test updates with invalid data** to ensure validation works



10. **Monitor duplicate key errors** (code 11000) separately

Need help with a specific update validation scenario or error handling pattern?